

Chapter 22 - Cameras and canvases

We use the code from the previous chapter

Camera

When you have a big level that doesn't fit the screen, we use a "camera" that follows the player around. Let's add a camera to the game we made in the previous chapter, where we have a player that collects coins.

So what does a camera do exactly? Basically it moves everything that we draw in a certain way so that the player is in the center. Now how would you solve this? We could have a variable like `camera_offset` and pass that to everything we're drawing.

```
love.graphics.circle("line", player.x - camera.offset.x, player.y - camera.offset.y, player.size)
```

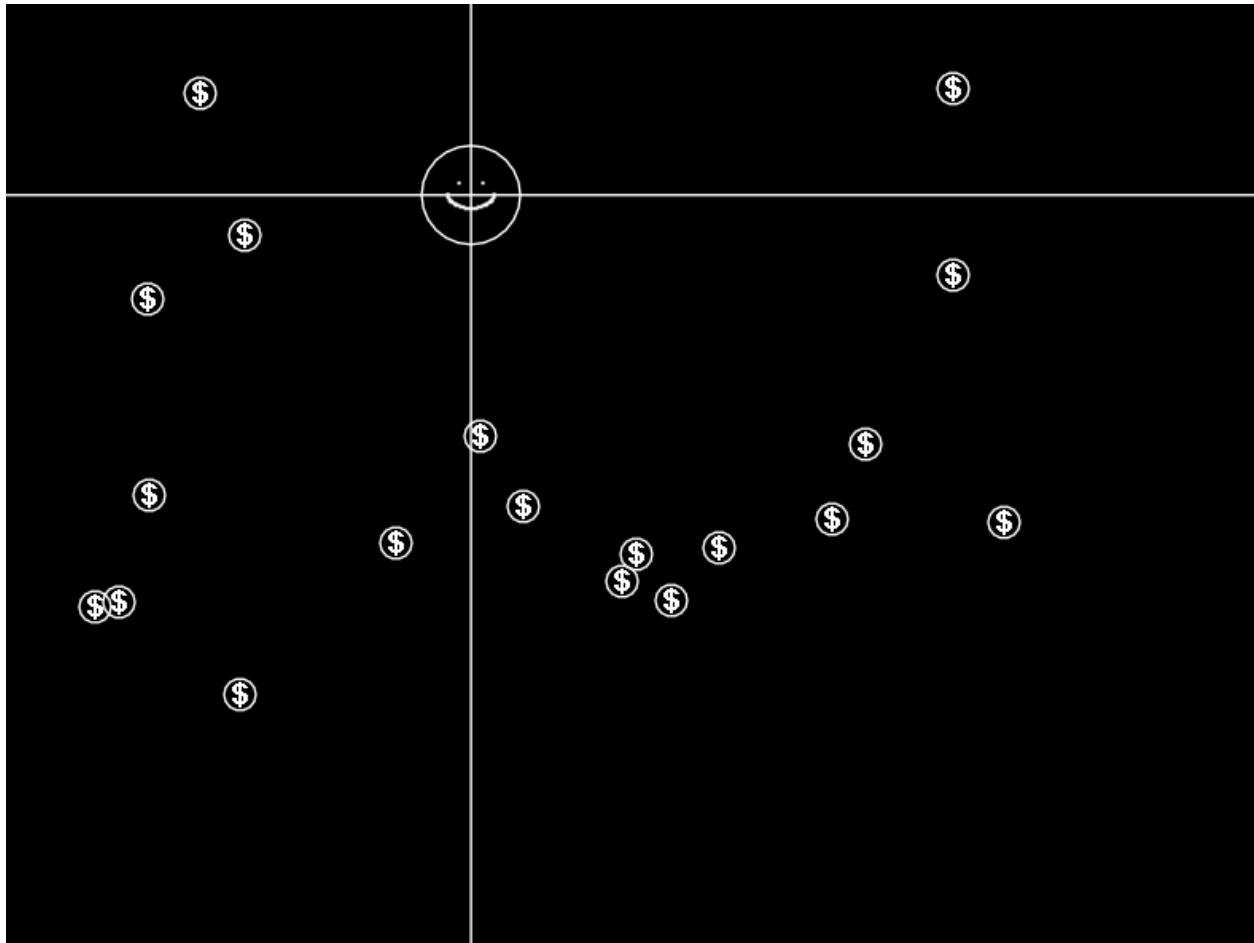
But if we have a very big game with a lot of things to draw, that's a lot of extra work. What we instead can do is use `love.graphics.translate()`. This function is part of the *Coordinate System*. Just like how you can move, rotate and scale an image, we can move, rotate and scale what we draw on. What we draw on is called the *canvas* (more on this later).

Normally when you draw something on position $(x=0, y=0)$, it will be drawn in the upper-left corner. With `love.graphics.translate(400, 200)` we move our canvas. Now if we draw something on $(0,0)$ it will be drawn on position $(400,200)$ instead. If we use `love.graphics.scale(2)` then everything we draw will be scaled up. Play around with it to really understand what it's doing.

So let's use `love.graphics.translate(x, y)` to create a camera. The question is in what way do we move the screen so that the player will appear in the center. First let's get the player located in the top-left of the screen. We do this by making the negative value of the player's position the $(0,0)$ point.

With the upper-left corner on $(0,0)$ (e.g. the default) the player is drawn on its position $(100, 50)$ like normal. But if we were to move that point with `love.graphics.translate(-100, -50)` it is now the player that is drawn in the upper-left corner. But we don't want it there, we want it in the center. So we add half the width and half the height of the screen (400 and 300).

A demonstration of what is happening. First we translate with `love.graphics.translate(-player.x, -player.y)` to get the player in the upper-left corner. Next we move the player to the center with `love.graphics.translate(-player.x + 400, -player.y + 300)`.



--code

function love.draw()

love.graphics.translate(-player.x + 400, -player.y + 300)

love.graphics.circle("line", player.x, player.y, player.size)

love.graphics.draw(player.image, player.x, player.y,

0, 1, 1, player.image:getWidth()/2, player.image:getHeight()/2)

for i,v in ipairs(coins) do

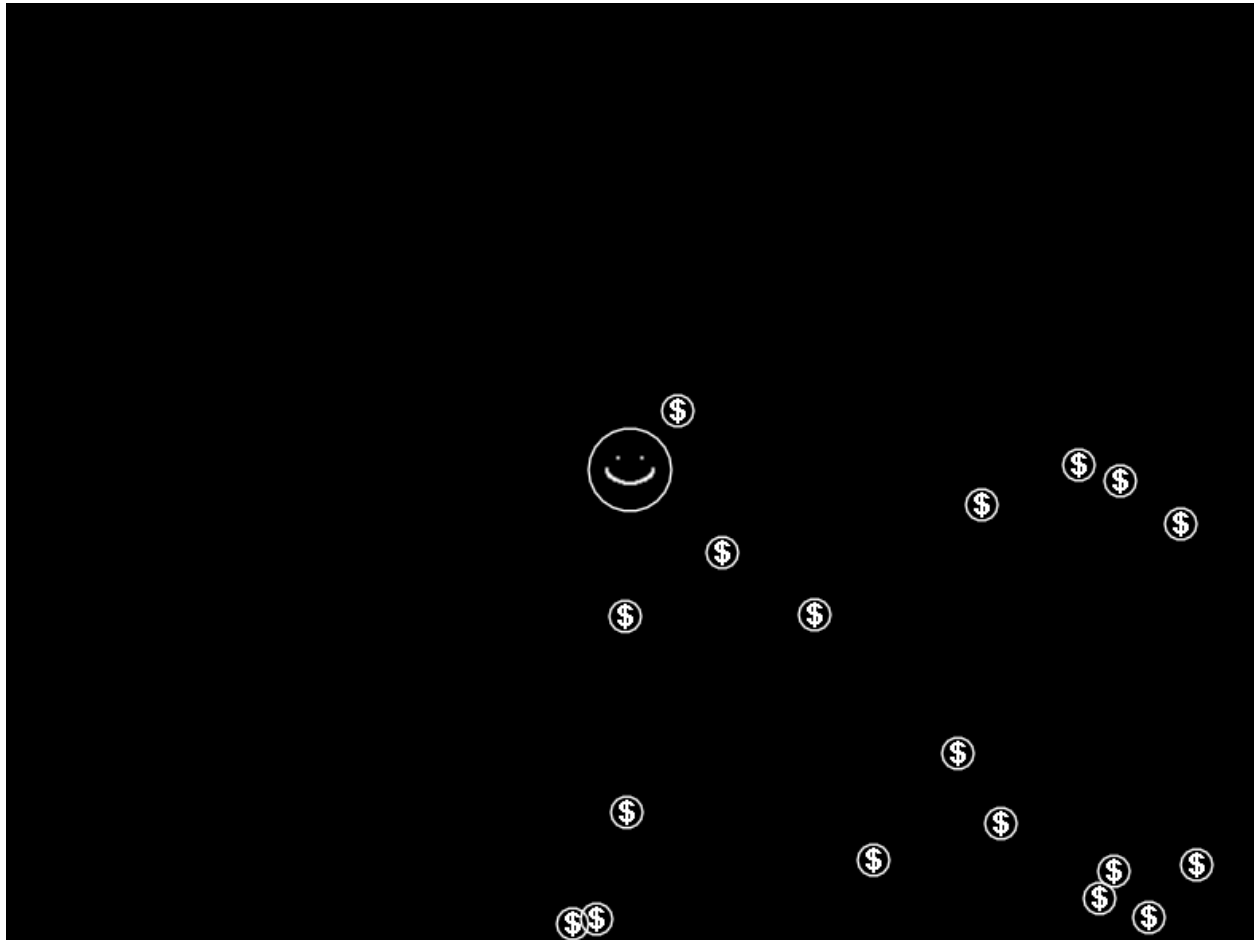
love.graphics.circle("line", v.x, v.y, v.size)

love.graphics.draw(v.image, v.x, v.y,

0, 1, 1, v.image:getWidth()/2, v.image:getHeight()/2)

end

end



It works! Now let's keep score of how many coins we have picked up. The score is something that you always want to see on screen. It's not part of the play area, it's part of the HUD. First let's add the score

```
-- In love.load()
```

```
score = 0
```

```
for i=#coins,1,-1 do
```

```
  if checkCollision(player, coins[i]) then
```

```
    table.remove(coins, i)
```

```
    player.size = player.size + 1
```

```
    score = score + 1
```

```
end
```

end

function love.draw()

```
love.graphics.translate(-player.x + 400, -player.y + 300)
love.graphics.circle("line", player.x, player.y, player.size)
love.graphics.draw(player.image, player.x, player.y,
    0, 1, 1, player.image:getWidth()/2, player.image:getHeight()/2)
```

for i,v in ipairs(coins) do

```
love.graphics.circle("line", v.x, v.y, v.size)
love.graphics.draw(v.image, v.x, v.y,
    0, 1, 1, v.image:getWidth()/2, v.image:getHeight()/2)
```

end

```
love.graphics.print(score, 10, 10)
```

end

When you run the game you can now see a number. Now how do we make it so that the score's position is not relative to the camera? One thing we could do is translate back to the normal position

function love.draw()

```
love.graphics.translate(-player.x + 400, -player.y + 300)
love.graphics.circle("line", player.x, player.y, player.size)
love.graphics.draw(player.image, player.x, player.y,
    0, 1, 1, player.image:getWidth()/2, player.image:getHeight()/2)
```

for i,v in ipairs(coins) do

```
love.graphics.circle("line", v.x, v.y, v.size)
love.graphics.draw(v.image, v.x, v.y,
    0, 1, 1, v.image:getWidth()/2, v.image:getHeight()/2)
```

end

```
love.graphics.translate(player.x - 400, player.y - 300)
```

```
love.graphics.print(score, 10, 10)
```

end

We can also use `love.graphics.origin()`. This resets all the coordinate transformations. This does not affect what you have already drawn.

function love.draw()

```
love.graphics.translate(-player.x + 400, -player.y + 300)
```

```
love.graphics.circle("line", player.x, player.y, player.size)
```

```
love.graphics.draw(player.image, player.x, player.y,
```

```
0, 1, 1, player.image:getWidth()/2, player.image:getHeight()/2)
```

for i,v in ipairs(coins) do

```
love.graphics.circle("line", v.x, v.y, v.size)
```

```
love.graphics.draw(v.image, v.x, v.y,
```

```
0, 1, 1, v.image:getWidth()/2, v.image:getHeight()/2)
```

end

```
love.graphics.origin()
```

```
love.graphics.print(score, 10, 10)
```

end

But this is not always the best option. Perhaps you want to scale your game, so you always want to have `love.graphics.scale(2)` and not have it reset. To fix that we can use `love.graphics.push()` and `love.graphics.pop()`. With `love.graphics.push()`, we save our current coordinate transformations and add it to a *stack*. With `love.graphics.pop()` we take the saved coordinate transformations from the top of the stack and apply it. Let's use these functions to draw our score correctly.

--- code

function love.draw()

```
love.graphics.push() -- Make a copy of the current state and push it onto the stack.
```

```
love.graphics.translate(-player.x + 400, -player.y + 300)
```

```
love.graphics.circle("line", player.x, player.y, player.size)
```

```

love.graphics.draw(player.image, player.x, player.y,
    0, 1, 1, player.image:getWidth()/2, player.image:getHeight()/2)
for i,v in ipairs(coins) do
    love.graphics.circle("line", v.x, v.y, v.size)
    love.graphics.draw(v.image, v.x, v.y,
        0, 1, 1, v.image:getWidth()/2, v.image:getHeight()/2)
end

love.graphics.pop() -- Pull the copy of the state of the stack and apply it.
love.graphics.print(score, 10, 10)
end

```

Nice, now we have a camera with a score that is drawn on top of the camera. Now let's make things shake.

Screenshake

How about adding some screenshake? First we need a timer for how long the shake needs to last and then make that timer go down.

```

-- In love.load()
shakeDuration = 0

-- In love.update(dt)
if shakeDuration > 0 then
    shakeDuration = shakeDuration - dt
end

```

Next we translate a few pixels in random directions as long as the timer is higher than 0.

```

function love.draw()
    love.graphics.push() -- Make a copy of the current state and push it onto the stack.
    love.graphics.translate(-player.x + 400, -player.y + 300)

```

if shakeDuration > 0 **then**

-- Translate with a random number between -5 and 5.

-- This second translate will be done based on the previous translate.

-- So it will not reset the previous translate.

love.graphics.translate(love.math.random(-5,5), love.math.random(-5,5))

end

love.graphics.circle("line", player.x, player.y, player.size)

love.graphics.draw(player.image, player.x, player.y,

0, 1, 1, player.image:getWidth()/2, player.image:getHeight()/2)

for i,v **in** ipairs(coins) **do**

love.graphics.circle("line", v.x, v.y, v.size)

love.graphics.draw(v.image, v.x, v.y,

0, 1, 1, v.image:getWidth()/2, v.image:getHeight()/2)

end

love.graphics.pop() *-- Pull the copy of the state of the stack and apply it.*

love.graphics.print(score, 10, 10)

end

And finally we set the timer to a certain value, for example 0.3, every time we pick up a coin.

for i=#coins,1,-1 **do**

if checkCollision(player, coins[i]) **then**

table.remove(coins, i)

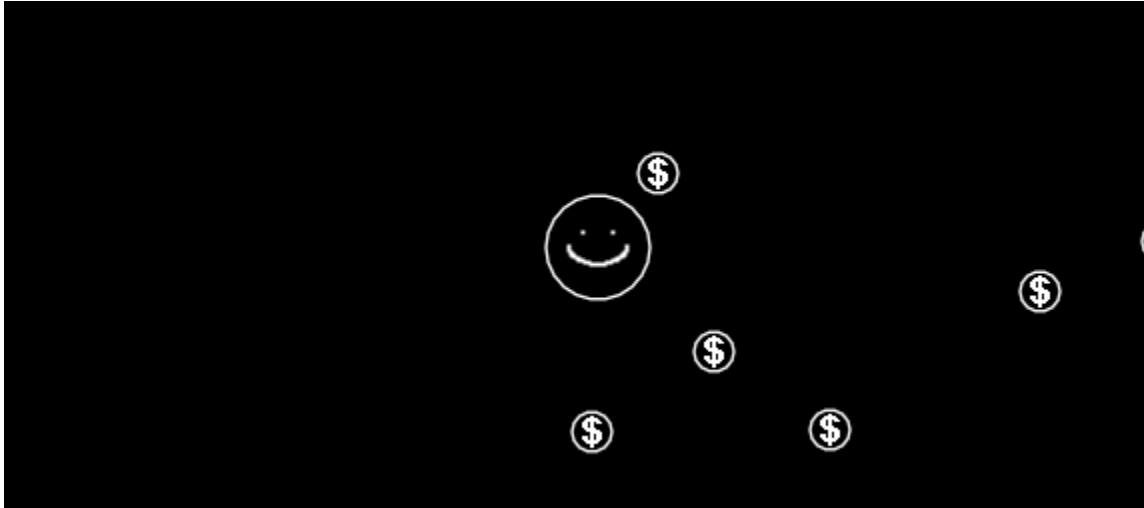
player.size = player.size + 1

score = score + 1

shakeDuration = 0.3

end

end



Try it out. It might be that the screen shakes too fast. And if not for you, it might happen on other computers. So again we need to use delta time. We can add a second timer to fix this. But then we also need an object to store our offset in.

```
-- In love.load()
```

```
shakeDuration = 0
```

```
shakeWait = 0
```

```
shakeOffset = {x = 0, y = 0}
```

```
-- In love.update(dt)
```

```
if shakeDuration > 0 then
```

```
    shakeDuration = shakeDuration - dt
```

```
    if shakeWait > 0 then
```

```
        shakeWait = shakeWait - dt
```

```
    else
```

```
        shakeOffset.x = love.math.random(-5,5)
```

```
        shakeOffset.y = love.math.random(-5,5)
```

```
        shakeWait = 0.05
```

```
end
```

```

end

function love.draw()
    love.graphics.push()

    love.graphics.translate(-player.x + 400, -player.y + 300)

    if shakeDuration > 0 then
        love.graphics.translate(shakeOffset.x, shakeOffset.y)
    end

    love.graphics.circle("line", player.x, player.y, player.size)

    love.graphics.draw(player.image, player.x, player.y,
        0, 1, 1, player.image:getWidth()/2, player.image:getHeight()/2)

    for i,v in ipairs(coins) do
        love.graphics.circle("line", v.x, v.y, v.size)

        love.graphics.draw(v.image, v.x, v.y,
            0, 1, 1, v.image:getWidth()/2, v.image:getHeight()/2)
    end

    love.graphics.pop()

    love.graphics.print(score, 10, 10)
end

```

And now it's consistent.

Now how about we make this game a multiplayer game, and add split screen?

Canvas

Whenever you use `love.graphics` to draw something, it is drawn on a *canvas*. This canvas is an object, and we can create our own canvas objects. This can be very useful, for a multiplayer game with split screen for example. You would draw the game on a separate canvas for each player, and then draw those canvases to the main canvas. Because a

canvas is a [Drawable](#). This is a superclass, and all LÖVE object that have this superclass can be drawn with `love.graphics.draw()`.

The supertypes of Canvas in order are:

- Texture
- Drawable
- Object

[Texture](#) is a subtype of Drawable. Both Image and Canvas are a subtype of Texture, and both can be drawn using a Quad. All LÖVE objects are of the type [Object](#). Let this not confuse you with the variable we use for classes, which we also name Object. These two are not related in any way, we just name them the same.

You can create a canvas with [love.graphics.newCanvas](#).

Let's use a canvas to create a split screen in our game. We start by adding another player that can move with WASD, and remove the screen shake and save/load system.

function love.load()

```
player1 = {  
    x = 100,  
    y = 100,  
    size = 25,  
    image = love.graphics.newImage("face.png")  
}
```

```
player2 = {  
    x = 300,  
    y = 100,  
    size = 25,  
    image = love.graphics.newImage("face.png")  
}
```

```
coins = {}
```

```
for i=1,25 do
```

```
    table.insert(coins,
```

```
        {
```

```
            x = math.random(50, 650),
```

```
            y = math.random(50, 450),
```

```
            size = 10,
```

```
            image = love.graphics.newImage("dollar.png")
```

```
        }
```

```
    )
```

```
end
```

```
score1 = 0
```

```
score2 = 0
```

```
end
```

```
function love.update(dt)
```

```
    if love.keyboard.isDown("left") then
```

```
        player1.x = player1.x - 200 * dt
```

```
    elseif love.keyboard.isDown("right") then
```

```
        player1.x = player1.x + 200 * dt
```

```
    end
```

```
    if love.keyboard.isDown("up") then
```

```

    player1.y = player1.y - 200 * dt
elseif love.keyboard.isDown("down") then
    player1.y = player1.y + 200 * dt
end

if love.keyboard.isDown("a") then
    player2.x = player2.x - 200 * dt
elseif love.keyboard.isDown("d") then
    player2.x = player2.x + 200 * dt
end

if love.keyboard.isDown("w") then
    player2.y = player2.y - 200 * dt
elseif love.keyboard.isDown("s") then
    player2.y = player2.y + 200 * dt
end

for i=#coins,1,-1 do
    if checkCollision(player1, coins[i]) then
        table.remove(coins, i)
        player1.size = player1.size + 1
        score1 = score1 + 1
    elseif checkCollision(player2, coins[i]) then
        table.remove(coins, i)
        player2.size = player2.size + 1
        score2 = score2 + 1
    end
end

```

end

end

end

function love.draw()

love.graphics.push()

love.graphics.translate(-player1.x + 400, -player1.y + 300)

love.graphics.circle("line", player1.x, player1.y, player1.size)

love.graphics.draw(player1.image, player1.x, player1.y,
0, 1, 1, player1.image:getWidth()/2, player1.image:getHeight()/2)

love.graphics.circle("line", player2.x, player2.y, player2.size)

love.graphics.draw(player2.image, player2.x, player2.y,
0, 1, 1, player2.image:getWidth()/2, player2.image:getHeight()/2)

for i,v in ipairs(coins) do

love.graphics.circle("line", v.x, v.y, v.size)

love.graphics.draw(v.image, v.x, v.y,
0, 1, 1, v.image:getWidth()/2, v.image:getHeight()/2)

end

love.graphics.pop()

love.graphics.print("Player 1 - " .. score1, 10, 10)

love.graphics.print("Player 2 - " .. score2, 10, 30)

end

```
function checkCollision(p1, p2)
```

```
-- Calculating distance in 1 line
```

```
-- Subtract the x's and y's, square the difference, sum the squares and find the root of the sum.
```

```
local distance = math.sqrt((p1.x - p2.x)^2 + (p1.y - p2.y)^2)
```

```
-- Return whether the distance is lower than the sum of the sizes.
```

```
return distance < p1.size + p2.size
```

```
end
```

Now we need a canvas to draw on. We're going to draw the canvas left and right, so it should be 400x600.

```
-- In love.load()
```

```
screenCanvas = love.graphics.newCanvas(400, 600)
```

Next we need to draw the game on to our new canvas. With `love.graphics.setCanvas()` we can set the canvas we want to draw on. If we pass no arguments to this function then it will reset to the default canvas.

```
function love.draw()
```

```
love.graphics.setCanvas(screenCanvas)
```

```
love.graphics.push()
```

```
love.graphics.translate(-player1.x + 400, -player1.y + 300)
```

```
love.graphics.circle("line", player1.x, player1.y, player1.size)
```

```
love.graphics.draw(player1.image, player1.x, player1.y,  
0, 1, 1, player1.image:getWidth()/2, player1.image:getHeight()/2)
```

```
love.graphics.circle("line", player2.x, player2.y, player2.size)
```

```
love.graphics.draw(player2.image, player2.x, player2.y,  
0, 1, 1, player2.image:getWidth()/2, player2.image:getHeight()/2)
```

```

for i,v in ipairs(coins) do

    love.graphics.circle("line", v.x, v.y, v.size)

    love.graphics.draw(v.image, v.x, v.y,
        0, 1, 1, v.image:getWidth()/2, v.image:getHeight()/2)

end

love.graphics.pop()

love.graphics.setCanvas()

love.graphics.print("Player 1 - " .. score1, 10, 10)

love.graphics.print("Player 2 - " .. score2, 10, 30)

```

end

If you run the game you'll see nothing. This is because the game is drawn onto our new canvas, but we're not drawing our canvas onto the default canvas. Like I said before, a Canvas is a Drawable and Drawable's can be drawn with love.graphics.draw.

function love.draw()

```

love.graphics.setCanvas(screenCanvas)

love.graphics.push()

love.graphics.translate(-player1.x + 400, -player1.y + 300)

love.graphics.circle("line", player1.x, player1.y, player1.size)

love.graphics.draw(player1.image, player1.x, player1.y,
    0, 1, 1, player1.image:getWidth()/2, player1.image:getHeight()/2)

love.graphics.circle("line", player2.x, player2.y, player2.size)

love.graphics.draw(player2.image, player2.x, player2.y,
    0, 1, 1, player2.image:getWidth()/2, player2.image:getHeight()/2)

```

```

for i,v in ipairs(coins) do

    love.graphics.circle("line", v.x, v.y, v.size)

    love.graphics.draw(v.image, v.x, v.y,

        0, 1, 1, v.image:getWidth()/2, v.image:getHeight()/2)

    end

love.graphics.pop()

love.graphics.setCanvas()

--Draw the canvas

love.graphics.draw(screenCanvas)

love.graphics.print("Player 1 - " .. score1, 10, 10)

love.graphics.print("Player 2 - " .. score2, 10, 30)

end

```

Now you should be able to see your game. But you'll notice something weird when you start moving. You're leaving a trail behind. This is because we are not clearing the canvas, meaning that everything we draw on it remains on the canvas. This is done by default on our default canvas. We can clear the canvas with `love.graphics.clear()`, and we should put this right before drawing the game. And since we're only using half of the screen, we need to reposition the camera to make the player in the center. Half the screen is now 200 instead of 400.

Since we're drawing the game twice, let's put everything we draw for our game in a function.

```

function love.draw()

    love.graphics.setCanvas(screenCanvas)

    love.graphics.clear()

    drawGame()

    love.graphics.setCanvas()

```

```
love.graphics.draw(screenCanvas)
```

```
love.graphics.setCanvas(screenCanvas)
```

```
    love.graphics.clear()
```

```
    drawGame()
```

```
love.graphics.setCanvas()
```

```
love.graphics.draw(screenCanvas, 400)
```

```
-- Add a line to separate the screens
```

```
love.graphics.line(400, 0, 400, 600)
```

```
love.graphics.print("Player 1 - " .. score1, 10, 10)
```

```
love.graphics.print("Player 2 - " .. score2, 10, 30)
```

```
end
```

```
function drawGame()
```

```
    love.graphics.push()
```

```
        love.graphics.translate(-player1.x + 200, -player1.y + 300)
```

```
    love.graphics.circle("line", player1.x, player1.y, player1.size)
```

```
    love.graphics.draw(player1.image, player1.x, player1.y,
```

```
        0, 1, 1, player1.image:getWidth()/2, player1.image:getHeight()/2)
```

```
    love.graphics.circle("line", player2.x, player2.y, player2.size)
```

```
    love.graphics.draw(player2.image, player2.x, player2.y,
```

```
        0, 1, 1, player2.image:getWidth()/2, player2.image:getHeight()/2)
```

```
for i,v in ipairs(coins) do
```

```
    love.graphics.circle("line", v.x, v.y, v.size)
```

```
    love.graphics.draw(v.image, v.x, v.y,
```

```
        0, 1, 1, v.image:getWidth()/2, v.image:getHeight()/2)
```

```
end
```

```
love.graphics.pop()
```

```
end
```

Now we have a split screen. The only problem is that both the left and the right screen focus on player 1. We need to make it so that the camera focuses on player 2 the second time we draw. We can fix this by adding a focus parameter to drawGame, and have the camera focus on what we pass as argument, which will be the players.

```
function love.draw()
```

```
    love.graphics.setCanvas(screenCanvas)
```

```
        love.graphics.clear()
```

```
        drawGame(player1)
```

```
    love.graphics.setCanvas()
```

```
    love.graphics.draw(screenCanvas)
```

```
    love.graphics.setCanvas(screenCanvas)
```

```
        love.graphics.clear()
```

```
        drawGame(player2)
```

```
    love.graphics.setCanvas()
```

```
    love.graphics.draw(screenCanvas, 400)
```

```
    love.graphics.line(400, 0, 400, 600)
```

```
love.graphics.print("Player 1 - " .. score1, 10, 10)
```

```
love.graphics.print("Player 2 - " .. score2, 10, 30)
```

```
end
```

```
function drawGame(focus)
```

```
love.graphics.push()
```

```
love.graphics.translate(-focus.x + 200, -focus.y + 300)
```

```
love.graphics.circle("line", player1.x, player1.y, player1.size)
```

```
love.graphics.draw(player1.image, player1.x, player1.y,  
0, 1, 1, player1.image:getWidth()/2, player1.image:getHeight()/2)
```

```
love.graphics.circle("line", player2.x, player2.y, player2.size)
```

```
love.graphics.draw(player2.image, player2.x, player2.y,  
0, 1, 1, player2.image:getWidth()/2, player2.image:getHeight()/2)
```

```
for i,v in ipairs(coins) do
```

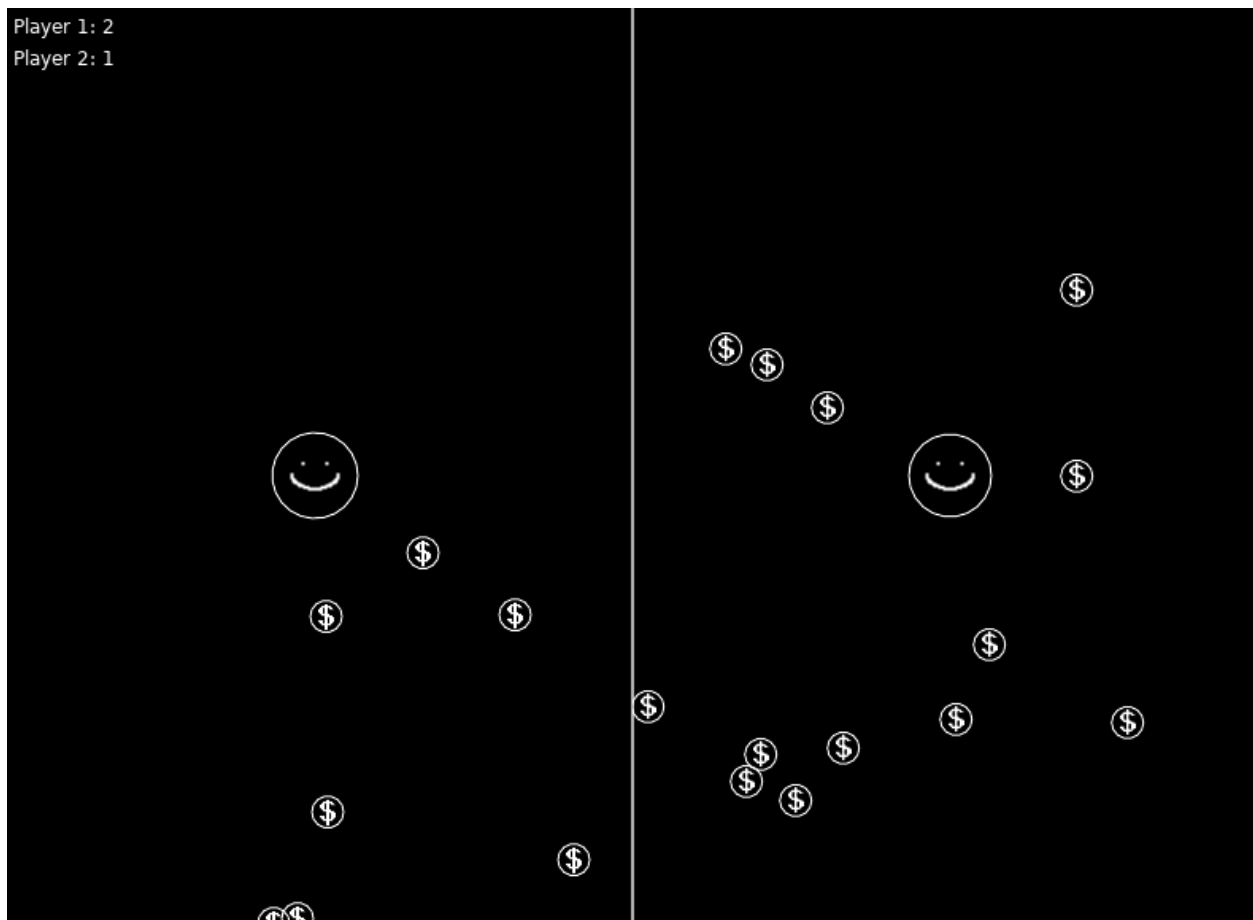
```
love.graphics.circle("line", v.x, v.y, v.size)
```

```
love.graphics.draw(v.image, v.x, v.y,  
0, 1, 1, v.image:getWidth()/2, v.image:getHeight()/2)
```

```
end
```

```
love.graphics.pop()
```

```
end
```



Libraries

We made a rather simple camera, but there's much more to it. Like zooming in and borders. I recommend checking out these libraries that provide advanced camera functionality:

- [Gamera](#)
- [Stalker-X](#)

Summary

With the Coordinate System we can change how stuff is drawn on screen. We can use `love.graphics.translate(x, y)` create a camera. Everything that we draw is drawn on a *canvas*. We can create our own canvas that we can draw like an image, because they're both of the LÖVE type *Texture*. By using a canvas we can create a split screen.